

Relatório de complexidade.

2ª Entrega: Estrutura de Dados.

Professor: Eduardo Savino

Grupo: José Aguiar, Luiza Alde, Bruno Faccio e Maria Luisa.

Informação do Projeto

A aplicação em Console foi desenvolvida em C# utilizando SQLite como banco de dados local. A aplicação possui como objetivo listar anúncios de produtos através de um sistema de paginação no console.

O projeto utiliza conceitos aprendidos em aula para organizar, buscar e exibir informações de maneira eficiente, evitando sobrecarga de processamento e melhorando a experiência do usuário.

Estruturas de Dados Utilizadas

Classe Produto

Foi criada uma estrutura de objeto chamada Produto, responsável por armazenar os dados dos anúncios.

Campos utilizados:

- Id
- Nome
- Preço
- Descrição
- Tipo
- Peso
- Condição

Essa estrutura organiza os dados de forma lógica e facilita o gerenciamento das informações dentro da aplicação.

Exemplo:

```
5 referências
public class Produto
{
    1 referência
    public int Id { get; set; }
    1 referência
    public string Nome { get; set; }

    1 referência
    public double Preco { get; set; }
    1 referência
    public string Descricao { get; set; }

    1 referência
    public string Tipo { get; set; }

    1 referência
    public string Peso { get; set; }

    1 referência
    public string Condicao { get; set; }
}
```

Busca de Dados

O sistema realiza buscas diretamente no banco SQLite utilizando comandos SQL.

A consulta principal utilizada foi:

```
SELECT * FROM anuncios LIMIT 3 OFFSET x
```

Ordenação de Dados

Exemplos de ordenações que podem ser aplicadas:

```
SELECT * FROM anuncios ORDER BY nome ASC
```

Ordena os produtos em ordem alfabética.

```
SELECT * FROM anuncios ORDER BY preco ASC
```

Ordena os produtos do menor para o maior preço.

```
SELECT * FROM anuncios ORDER BY preco DESC
```

Ordena os produtos do maior para o menor preço.

SELECT * FROM anuncios WHERE tipo = 'Grãos'

Realiza uma busca filtrando os produtos por categoria.

As ordenações foram realizadas utilizando o comando:

ORDER BY

Complexidade da ordenação de dados.

A ordenação depende do banco de dados SQLite, que normalmente utiliza algoritmos otimizados internamente.

Complexidade média:

- Ordenação: $O(n \log n)$

Onde:

- n = quantidade de registros.

Essa abordagem permite que os dados sejam exibidos de maneira organizada sem necessidade de ordenar manualmente no código.

Complexidade da busca de dados.

A busca é feita utilizando paginação.

Isso evita carregar todos os registros de uma única vez na memória, reduzindo o consumo de recursos da aplicação.

Complexidade aproximada:

- Busca paginada: $O(k)$
- Onde:
 - k = quantidade de registros carregados por página.

Como apenas 3 registros são carregados por vez, o desempenho permanece estável mesmo com crescimento do banco de dados e a implementação de novos produtos.

Paginação

A paginação foi implementada utilizando:

- LIMIT (define quantos registros serão exibidos)
- OFFSET (define quantos registros serão ignorados antes da leitura)

E uma variável de controle:

```
0 referências
class Program
{
    static int paginaAtual = 0;
```

Exemplo de LIMIT.

```
int limite = 3;
int pular = paginaAtual * limite;
```

Essas variáveis melhora a eficiência da aplicação e execução da busca.

Estruturas de Controle

Foram utilizadas estruturas de repetição e decisão para controlar a navegação do sistema.

Estruturas utilizadas

Repetição

while (rodando)

Responsável por manter o sistema em execução até o usuário escolher sair.

Decisão

if

else if

Responsáveis por controlar:

- Próxima página
- Página anterior
- Encerramento da aplicação

Otimizações Aplicadas.

Paginação

A principal otimização do sistema foi a paginação dos dados.

Sem paginação:

- Todos os anúncios seriam carregados simultaneamente.
- Maior consumo de memória.
- Possível lentidão.

Com paginação:

- Apenas os registros necessários são carregados.
- Menor uso de memória.
- Melhor desempenho.

Uso do SQLite

O SQLite foi utilizado por ser leve, eficiente e também um pouco mais simples de implementar dentro do código, apenas necessitando de um pacote NuGet.

Benefícios:

- Fácil integração.
- Baixo consumo de recursos.
- Boa velocidade em consultas simples.

Conclusão

O projeto aplicou conceitos importantes de Estrutura de Dados através da organização de objetos, controle de fluxo e otimização de consultas, a utilização da paginação permitiu melhorar o desempenho da aplicação, evitando carregamentos desnecessários e tornando a navegação mais eficiente, além disso, o uso de classes para representar os produtos contribuiu para uma estrutura de código mais organizada, reutilizável e de fácil manutenção. Dessa forma, o sistema conseguiu aplicar conceitos de Estrutura de Dados tanto na organização das informações quanto na otimização do processamento da aplicação.